

Συστήματα Υπολογισμού Υψηλών Επιδόσεων Project 1

Τσόγκας Παναγιώτης	Νίκας Ιωάννης Ιάσων	Τσιαντός Δημήτριος
ptsogkas@uth.gr	ioannikas@uth.gr	dtsiantos@uth.gr
3672	3771	3796

Οκτώβριος 2025

Περιεχόμενα

1	Εισαγωγή	3
2	Συνηθισμένες βελτιστοποιήσεις	4
2.1	Loop interchange	4
2.2	Loop unrolling	5
2.3	Loop fusion	7
2.4	Function inlining	7
2.5	Common sub-expression elimination & Loop Invariant code motion	8
2.6	Strength reduction	9
2.7	Referencing	10
3	Μαθηματικές βελτιστοποιήσεις	10
3.1	Baseline	10
3.2	Γρήγορο Convolution	11
3.3	ή PSNR	12
3.4	Γρήγορη min συνάρτηση	12
3.5	Γρήγορη Square Root	13
4	Αποτελέσματα	16

1 Εισαγωγή

Έχοντας μία αρχική υλοποίηση του τελεστή Sobel σε γλώσσα προγραμματισμού C προσπαθούμε να προσαρμόσουμε τον κώδικα στα χαρακτηριστικά του συστήματος με σκοπό τον χαμηλότερο δυνατό χρόνο εκτέλεσης. Προκειμένου να επιτευχθεί αυτό, μπορούμε να εφαρμόσουμε διάφορες μεθόδους βελτιστοποίησης που λειτουργούν τόσο σε επεξεργαστές γενικού σκοπού, όσο και συγκεκριμένα στην μικροαρχιτεκτονική του συστήματός μας.

Name	12th Gen Intel® Core™ i5-1235U × 12
Cache	12 MB Intel® Smart Cache
RAM	16GB DDR4
OS	Ubuntu 22.04.5 LTS
Kernel	6.8.0-85-generic
Compiler	Intel(R) oneAPI DPC++/C++ Compiler 2025.2.1
Compiler Version	2025.2.0.20250806

Table 1: Χρήσιμα χαρακτηριστικά του συστήματος.

Για την δοκιμή του αλγορίθμου χρησιμοποιούμε ως είσοδο εικόνα μεγέθους 4096×4096 pixels. Εκτελούμε τον αλγόριθμο 12 φορές και αγνοούμε από τον μέσο όρο την πιο γρήγορη και πιο αργή εκτέλεση. Όλες οι εκτελέσεις γίνονται σε ένα Performance-core του επεξεργαστή (4.4GHz max).

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
01_sobel_orig	1.00×	1.75385	0.04514	1.690210 / 1.916120

Πίνακας 2: Χρόνος εκτέλεσης σε -O0

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
01_sobel_orig	1.00×	0.46939	0.01264	0.450155 / 0.518506

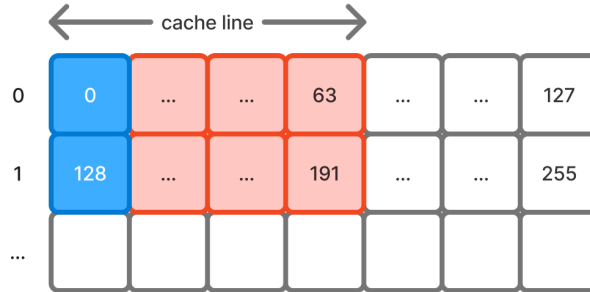
Πίνακας 3: Χρόνος εκτέλεσης σε -fast

2 Συνηθισμένες βελτιστοποιήσεις

2.1 Loop interchange

Παρατηρούμε πως η αρχική υλοποίηση διατρέχει τα στοιχεία της κάθε στήλης. Αυτός ο τρόπος προσπέλασης των στοιχείων είναι πολύ αργός καθώς δεν εκμεταλλεύεται τον μηχανισμό προανάκτησης (prefetching) του επεξεργαστή. Σε κάθε προσπέλαση μνήμης, ο επεξεργαστής φέρνει μία ολόκληρη γραμμή (cache line) στην cache, θεωρώντας πως τα δεδομένα που είναι σχετικά μεταξύ τους αποθηκεύονται σε διαδοχικές θέσεις στην μνήμη.

Διαβάζοντας το πρώτο στοιχείο της πρώτης σειράς του πίνακα και ύστερα το πρώτο στοιχείο της δεύτερης σειράς, φορτώνονται και η πρώτη και η δεύτερη σειρά στην cache, χωρίς ωστόσο να τις χρησιμοποιήσουμε πριν προεκτοπιστούν.



Σχήμα 1: Προσπέλαση στοιχείων του πίνακα ανά στήλη. Τα κόκκινα στοιχεία τοποθετούνται στην cache αλλά δεν θα προσπελαστούν άμεσα από τον επεξεργαστή.

Το αποτέλεσμα είναι να δημιουργούνται πολλά cache misses και κατ' επέκταση stalls, καθυστερώντας κατά πολύ την εκτέλεση του προγράμματος. Αλλάζοντας το access pattern σε διαδοχικές θέσεις μνήμης, μειώνεται σημαντικά ο χρόνος εκτέλεσης, καθώς μειώνονται τα cache misses.

```
/* For each pixel of the output image */
for (i=1; i<SIZE-1; i+=1 ) {
    for (j=1; j<SIZE-1; j+=1) {
        /* ... */
    }
}
```

Metric	Original	Interchange	Difference
Cache misses	57.7M	2.6M	22×
Cycles	7.4B	5.32B	39%
IPC (instr./cycle)	2.44	3.39	+0.95

Πίνακας 4: Σύγκριση σε -O0.

Metric	Original	Interchange	Difference
Cache misses	32.3M	2.65M	12.2×
Cycles	2.02B	0.854B	58%
IPC (instr./cycle)	0.76	1.80	+1.04

Πίνακας 5: Σύγκριση σε -fast.

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
01_sobel_orig	1.00×	1.75385	0.04514	1.690210 / 1.916120
02_sobel_loop_interchange	1.44×	1.22102	0.01533	1.193020 / 1.259970

Πίνακας 6: Χρόνος εκτέλεσης σε-00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
01_sobel_orig	1.00×	0.46939	0.01264	0.450155 / 0.518506
02_sobel_loop_interchange	2.79×	0.16796	0.00047	0.167034 / 0.177655

Πίνακας 7: Χρόνος εκτέλεσης σε-fast

2.2 Loop unrolling

Όταν έχουμε ένα loop, η διαχείρησή προσθέτει overhead στο σύστημα, καθώς σε κάθε επανάληψη θα πρέπει να εκτελεστεί μία συνθήκη για τον έλεγχο τερματισμού και μία πράξη για την αλλαγή του δείκτη. Αυτές οι πράξεις μπορούν να μειωθούν με το unroll.

```
for (i = 1; i < SIZE - 1; i++) {
    for (j = 1; j < SIZE - 1 - remainder; j += 4) {
        // pixel (i, j)
        p = ...;
        res = ...;
        output[i*SIZE + j] = ...;

        // pixel (i, j+1)
        p = ...;
        res = ...;
        output[i*SIZE + j+1] = ...;

        // pixel (i, j+2)
        p = ...;
        res = ...;
        output[i*SIZE + j+2] = ...;

        // pixel (i, j+3)
        p = ...;
        res = ...;
        output[i*SIZE + j+3] = ...;
    }
}
```

Στο συγκεκριμένο παράδειγμα ο παράγοντας unroll είναι 4. Ο αριθμός των branches και οι προσθέσεις του loop υποτετραπλασιάζονται. Από τις μετρήσεις του perf παρατηρούμε πως στην εκτέλεση χωρίς βελτιστοποιήσεις μεταγλωττιστή ο αριθμός των συνολικών branches μειώθηκε σημαντικά. Στην περίπτωση της εκτέλεσης με -O3, ο μεταγλωττιστής βελτιστοποιεί τις διαδοχικές εντολές εντός του loop, κάνοντας έτσι την εκτέλεση ακόμα πιο γρήγορη.

Metric	Interchange	Unrolled	Difference
Branch instructions	2B	987M	2.1×
Cycles	5.3B	4B	1.31×
IPC (instr./cycle)	3.41	3.1	-0.4

Πίνακας 8: Σύγκριση σε -O0.

Metric	Interchange	Unrolled	Difference
Branch instructions	42.5M	36.8M	1.16×
Cycles	854M	599M	1.43×
IPC (instr./cycle)	1.80	2.62	+0.82

Πίνακας 9: Σύγκριση σε -fast.

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
02_sobel_loop_interchange	1.44×	1.22102	0.01533	1.193020 / 1.259970
03_sobel_loop_unrolling	1.94×	0.90287	0.00164	0.900786 / 0.942234

Πίνακας 10: Χρόνος εκτέλεσης σε -O0

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
02_sobel_loop_interchange	2.79×	0.16796	0.00047	0.167034 / 0.177655
03_sobel_loop_unrolling	4.25×	0.11050	0.00047	0.108389 / 0.112336

Πίνακας 11: Χρόνος εκτέλεσης σε -fast

2.3 Loop fusion

Για υπολογισμό του PSNR του αποτελέσματος, διατρέχονται σε νέο loop η εικόνα που παράχθηκε από το πρώτο loop και η σωστή (golden) εικόνα. Αντί για δύο διπλά loops, θα μπορούσαμε να ενσωματώσουμε όλους τους υπολογισμούς σε ένα loop, μειώνοντας τον συνολικό αριθμό επαναλήψεων και εντολών.

```
for (i = 1; i < SIZE - 1; i++) {
    for (j = 1; j < SIZE - 1 - remainder; j += 4) {
        // pixel (i, j)
        p = ...
        res = ...
        output[i*SIZE + j] = ...
        t = pow((output[i*SIZE+j] - golden[i*SIZE+j]),2);
        PSNR += t;

        // pixel (i, j+1)
        p = ...
        res = ...
        output[i*SIZE + j+1] = ...
        t = pow((output[i*SIZE+j+1] - golden[i*SIZE+j+1]),2);
        PSNR += t;

        /* ... */
    }
}
```

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
03_sobel_loop_unrolling	1.94×	0.90287	0.00164	0.900786 / 0.942234
04_sobel_loop_fusion	2.14×	0.82101	0.01090	0.810427 / 0.964667

Πίνακας 12: Χρόνος εκτέλεσης σε-00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
03_sobel_loop_unrolling	4.25×	0.11050	0.00047	0.108389 / 0.112336
04_sobel_loop_fusion	4.35×	0.10795	0.00071	0.107109 / 0.109979

Πίνακας 13: Χρόνος εκτέλεσης σε-fast

2.4 Function inlining

Η κλήση συναρτήσεων στην C είναι μία διαδικασία που έχει overhead από διαδικασίες όπως δέσμευση της stack, αποθήκευση ορισμάτων, διεύθυνσης επιστροφής, τοπικές μεταβλητές, την διαγραφή αυτών στο τέλος της συνάρτησης καθώς και τον μηχανισμό επιστροφής. Στον κώδικά μας καλείται η συνάρτηση convolution2d επαναληπτικά, κάνοντας το overhead των κλήσεων να γίνεται αισθητό. Για να το λύσουμε αυτό μπορούμε να αλλάξουμε την συνάρτηση ώστε να είναι macro που θα αντικαταστήσει αργότερα ο preprocessor όπου χρειάζεται.

```
#define CONVOLUTION2D(posy, posx, input, operator)
```

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
04_sobel_loop_fusion	2.14×	0.82101	0.01090	0.810427 / 0.964667
05_sobel_function_inlining	2.28×	0.76785	0.01184	0.753455 / 0.798266

Πίνακας 14: Χρόνος εκτέλεσης σε-00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
04_sobel_loop_fusion	4.35×	0.10795	0.00071	0.107109 / 0.109979
05_sobel_function_inlining	4.38×	0.10729	0.00080	0.106468 / 0.119504

Πίνακας 15: Χρόνος εκτέλεσης σε-fast

2.5 Common sub-expression elimination & Loop Invariant code motion

Μια ακόμη σημαντική τεχνική βελτιστοποίησης είναι η εξάλειψη κοινών υποεκφράσεων (Common Subexpression Elimination), η οποία αποσκοπεί στην αποφυγή επαναλαμβανόμενων υπολογισμών της ίδιας μαθηματικής έκφρασης μέσα σε έναν βρόχο ή τμήμα κώδικα.

Παρόμοια λειτουργία επιτυγχάνει το Loop Invariant Code Motion, κατά το οποίο ένα κομμάτι κώδικα που δεν εξαρτάται από τις μεταβλητές του βρόχου μετακινείται εκτός του loop και υπολογίζεται μία φορά.

```
for (i = 1; i < SIZE - 1; i++) {
    /* Calculate expression once, refer to it by variable i_times_SIZE */
    i_times_SIZE = i * SIZE;
    for (j = 1; j < SIZE - 1 - remainder; j += unroll_factor) {
        // pixel (i, j)
        p = ...;
        res = ...;

        /* Refer by variable to avoid redundant additions*/
        i_times_SIZE_plus_j = i_times_SIZE + j;
        output[i_times_SIZE_plus_j] = ...;

        /* Store intermediate result in t to prevent recalculating */
        t = pow((output[i_times_SIZE_plus_j] - golden[i_times_SIZE_plus_j]), 2);
        PSNR += t;

        // pixel (i, j+1)
        ...

        /* ... */
    }
}
```

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
05_sobel_function_inlining	2.28×	0.76785	0.01184	0.753455 / 0.798266
06_sobel_common_subexpression_elimination	1.95×	0.89913	0.01290	0.890251 / 0.958620

Πίνακας 16: Χρόνος εκτέλεσης σε-00

Όπως παρατηρούμε, η εκτέλεση είναι πιο αργή σε σχέση με την προηγούμενη έκδοση. Παρ' όλα αυτά, η εφαρμογή των δύο αυτών βελτιστοποιήσεων δημιουργεί τη βάση για περαιτέρω βελτίωση της απόδοσης σε μεταγενέστερο στάδιο της υλοποίησης.

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
05_sobel_function_inlining	4.38×	0.10729	0.00080	0.106468 / 0.119504
06_sobel_common_subexpression_elimination	4.12×	0.11386	0.00643	0.105915 / 0.162000

Πίνακας 17: Χρόνος εκτέλεσης σε-fast

2.6 Strength reduction

Η χρήση αργών τελεστών μπορεί να επιβαρύνει σημαντικά την απόδοση του προγράμματος, ιδίως όταν αυτοί χρησιμοποιούνται εντός του βασικού loop. Για τον λόγο αυτό, η αντικατάσταση πολύπλοκων πράξεων με απλούστερους τελεστές ή πιο αποδοτικές υλοποιήσεις, μπορεί να μειώσει αισθητά το computational load του προγράμματος.

```
for (i = 1; i < SIZE - 1; i++) {
    for (j = 1; j < SIZE - 1 - remainder; j += unroll_factor) {
        // pixel (i, j)
        pixel_horizontal = ...;
        pixel_vertical = ...;
        /* Remove pow use multiplication instead */
        res = sqrt(pixel_horizontal * pixel_horizontal + pixel_vertical * pixel_vertical);
        i_times_SIZE_plus_j = ...;
        output[i_times_SIZE_plus_j] = ...;
        t = ...;
        PSNR += ...;

        // pixel (i, j+1)
        /* ... */
    }
    /* Eliminate multiplication use addition instead */
    i_times_SIZE += SIZE;
}
```

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
06_sobel_common_subexpression_elimination	1.95×	0.89913	0.01290	0.890251 / 0.958620
07_sobel_strength_reduction	8.30×	0.21133	0.00381	0.208410 / 0.226750

Πίνακας 18: Χρόνος εκτέλεσης σε-00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
06_sobel_common_subexpression_elimination	4.12×	0.11386	0.00643	0.105915 / 0.162000
07_sobel_strength_reduction	5.02×	0.09359	0.00079	0.092326 / 0.095376

Πίνακας 19: Χρόνος εκτέλεσης σε-fast

2.7 Referencing

Το referencing χρησιμοποιείται για την αποφυγή περιττών πράξεων δεικτών με σταθερές τιμές, οι οποίες μπορούν να αποφευχθούν μέσω της επαναπροσδιορισμού του δείκτη με το κατάλληλο offset. Επιπλέον, η τεχνική αυτή διευκολύνει τη χρήση pointer arithmetic, ωστόσο παρατηρήθηκε ότι είχε αρνητικό αντίκτυπο στον χρόνο εκτέλεσης, καθώς δεν επέτρεπε τη χρήση του keyword const. Για τον λόγο αυτό, η συγκεκριμένη προσέγγιση δεν συμπεριλήφθηκε στις βελτιστοποιήσεις.

```
for (i = 1; i < SIZE - 1; i++) {
    /* Referencing array offset address */
    const unsigned char *top_row = &input[dec_i_times_SIZE];
    const unsigned char *bottom_row = &input[inc_i_times_SIZE];
    const unsigned char *mid_row = &input[i_times_SIZE];
    unsigned char *out_row = &output[i_times_SIZE];
    unsigned char *gold_row = &golden[i_times_SIZE];
    for (j = 1; j < SIZE - 1 - remainder; j += unroll_factor) {
        /* Refer to convolution inputs by pointers avoid redundant operations */
        pixel_horizontal = CONVOLUTION2D(j, top_row, mid_row, bottom_row, horiz_operator);
        pixel_vertical = CONVOLUTION2D(j, top_row, mid_row, bottom_row, vert_operator);
        res = ...;
        out_row[j] = ...;
        t = (out_row[j] - gold_row[j]);
        PSNR += t * t;

        /* ... */
    }
    dec_i_times_SIZE += SIZE;
    inc_i_times_SIZE += SIZE;
    i_times_SIZE += SIZE;
}
```

3 Μαθηματικές βελτιστοποιήσεις

3.1 Baseline

Σαν base version επιλέξαμε την ταχύτερη υλοποίηση σε single core με infinite PSNR, όμως αφαιρέσαμε το βήμα του loop fusion, καθώς παρατηρήσαμε ότι το πρόγραμμα ήταν ιδιαίτερα memory-bound στα μεταγενέστερα version του κώδικα, λόγω της αύξημενης απόδοσης από τα computational optimizations και των εναλλασσόμενων προσπελάσεων στους πίνακες golden & output. Επιπλέον επιλογή ήταν η αφαίρεση του loop unrolling που επέτρεψε την ευκολότερη απομόνωση μελλοντικών προβλημάτων με χρήση V-Tune, διευκολύνοντας γενικά την ανάλυση και συγκεκριμένα την ανάλυση της συμπεριφοράς της μνήμης. Η αφαίρεση των δύο βελτιστοποιήσεων οδήγησε αργότερα στους καλύτερους χρόνους εκτέλεσης σε σχέση με άλλα base versions.

3.2 Γρήγορο Convolution

Η πράξη της συνέλιξης (convolution) αποτελεί μία από τις πιο υπολογιστικά απαιτητικές πράξεις. Ωστόσο, αξιοποιώντας τις ιδιότητες του συγκεκριμένου kernel, είναι δυνατόν να απλοποιηθεί η πράξη της συνέλιξης μέσω κατάλληλων μαθηματικών μετασχηματισμών, μειώνοντας έτσι τον συνολικό αριθμό πράξεων και βελτιώνοντας την απόδοση.

Επιπλέον, ιδιαίτερη σημασία έχει η σειρά εκτέλεσης των πράξεων, καθώς η οργάνωση των υπολογισμών με τρόπο που ακολουθεί ένα προβλέψιμο μοτίβο πρόσβασης στη μνήμη συμβάλλει στη βελτίωση του cache locality. Με αυτόν τον τρόπο, πραγματοποιούνται όλες οι απαραίτητες πράξεις ανά γραμμή του kernel, ελαχιστοποιώντας τα cache misses και evictions χρήσιμων δεδομένων, γεγονός που οδηγεί σε ταχύτερη εκτέλεση.

```
for (i = 1; i < SIZE - 1; i++) {
    const unsigned char *top_row = &input[dec_i_times_SIZE];
    const unsigned char *bot_row = &input[inc_i_times_SIZE];
    const unsigned char *mid_row = &input[i_times_SIZE];
    unsigned char *out_row = &output[i_times_SIZE];

    for (j = 1; j < SIZE - 1; j++) {
        /* Convolution computations row by row */
        pixel_horizontal = -top_row[j - 1] + top_row[j + 1];
        pixel_vertical = top_row[j - 1] + (top_row[j] << 1) + top_row[j + 1];
        pixel_horizontal += -(mid_row[j - 1] << 1) + (mid_row[j + 1] << 1) -
            bot_row[j - 1] + bot_row[j + 1];
        pixel_vertical += -bot_row[j - 1] - (bot_row[j] << 1) -
            bot_row[j + 1];

        /* ... */
    }
    dec_i_times_SIZE += SIZE;
    inc_i_times_SIZE += SIZE;
    i_times_SIZE += SIZE;
}
```

Αυτό προκύπτει από:

$$H = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$\begin{aligned} \text{pixel_horizontal} &= (-1) \cdot \text{top_row}[j-1] + 0 \cdot \text{top_row}[j] + 1 \cdot \text{top_row}[j+1] \\ &\quad + (-2) \cdot \text{mid_row}[j-1] + 0 \cdot \text{mid_row}[j] + 2 \cdot \text{mid_row}[j+1] \\ &\quad + (-1) \cdot \text{bot_row}[j-1] + 0 \cdot \text{bot_row}[j] + 1 \cdot \text{bot_row}[j+1] \end{aligned}$$

Αφαιρώντας τους όρους που πολλαπλασιάζονται με το μηδέν:

$$\text{pixel_horizontal} = -\text{top_row}[j-1] + \text{top_row}[j+1] - 2 \cdot \text{mid_row}[j-1] + 2 \cdot \text{mid_row}[j+1] - \text{bot_row}[j-1] + \text{bot_row}[j+1]$$

Με τον ίδιο τρόπο βγάζουμε και το pixel_vertical.

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
09_sobel_no_unroll_no_fusion	12.08×	0.14522	0.00156	0.143731 / 0.151505
10_sobel_fast_convolution	14.53×	0.12072	0.00102	0.119427 / 0.126503

Πίνακας 20: Χρόνος εκτέλεσης σε-00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
09_sobel_no_unroll_no_fusion	6.11×	0.07683	0.00069	0.075577 / 0.079137
10_sobel_fast_convolution	17.30×	0.02714	0.00057	0.026333 / 0.028785

Πίνακας 21: Χρόνος εκτέλεσης σε-fast

3.3 Γρήγορο PSNR

Εάν m και n οι διαστάσεις της εικόνας, I το αποτέλεσμα του αλγορίθμου και K η ιδανική εικόνα, τότε το MSE υπολογίζεται ως:

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I_{i,j} - K_{i,j}]^2$$

Για τον υπολογισμό αυτόν όμως, θα χρειαστεί να εκτελέσουμε είτε στο βασικό είτε σε δεύτερο loop την πράξη $[I_{i,j} - K_{i,j}]^2$. Αν η πράξη αυτή γίνει στο βασικό loop τότε θα φέρνουμε στην cache στοιχεία του πίνακα K γεμίζοντάς την ενώ αν γίνει σε ξεχωριστό loop θα υπάρχει μεγάλο overhead. Για να αποφύγουμε αυτή την πράξη, θα μπορούσαμε να μετασχηματίσουμε τον τύπο του MSE ώστε να κάνουμε προσπελάσεις στο κύριο loop μόνο στον πίνακα I .

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I_{i,j}^2 - 2I_{i,j}K_{i,j} + K_{i,j}^2] = \frac{1}{mn} (I^2 - 2IK + K^2)$$

Τώρα μπορούμε να υπολογίσουμε σε μία μεταβλητή τον όρο I^2 και ύστερα σε δεύτερο loop να υπολογίσουμε τους όρους $-2IK + K^2$.

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
10_sobel_fast_convolution	14.53×	0.12072	0.00102	0.119427 / 0.126503
11_sobel_fast_psnr	16.30×	0.10762	0.00150	0.104974 / 0.111273

Πίνακας 22: Χρόνος εκτέλεσης σε-00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
10_sobel_fast_convolution	17.30×	0.02714	0.00057	0.026333 / 0.028785
11_sobel_fast_psnr	13.18×	0.03562	0.00044	0.034291 / 0.037935

Πίνακας 23: Χρόνος εκτέλεσης σε-fast

3.4 Γρήγορη min συνάρτηση

Μέσα στο βασικό loop ο αλγόριθμος παίρνει το αποτέλεσμα της square root και αν αυτό είναι μεγαλύτερο από 255 το θέτει ίσο με 255, αλλιώς κρατάει την τιμή επιστροφής της square root. Αυτή είναι η λογική της συνάρτησης min, καθώς βρίσκει το ελάχιστο μεταξύ του 255 και την επιστροφή της square root.

$$output_{i,j} = \min(255, (int)sqrt(pixel))$$

Η αρχική υλοποίηση της min είναι με if-else statement, αυξάνοντας τις εντολές διακλάδωσης στο πρόγραμμα. Για να το αποφύγουμε αυτό, χρησιμοποιήσαμε μία εκδοχή της συνάρτησης min που δεν έχει καμία εντολή διακλάδωσης.

$$diff = pixel - 255$$

$$output_{i,j} = 255 + diff \& (diff >> 31)$$

Σε αυτή την συνάρτηση όταν το pixel είναι μικρότερο του 255, ο όρος diff είναι αρνητικός και ύστερα όταν γίνεται shift δεξιά κατά 31 bits, η μάσκα γίνεται 0xFFFFFFFF λόγω του sign extension. Έτσι, όταν μειώνουμε το diff από το 255 έχουμε ως αποτέλεσμα την αρχική τιμή του pixel.

$$diff = x - 255, \quad x < 255$$

$$output_{i,j} = 255 + diff \& 0xFFFFFFFF$$

Από την πράξη $diff \& 0xFFFFFFFF$ παίρνουμε $diff$, άρα

$$output_{i,j} = 255 + diff = 255 + pixel - 255 = pixel$$

Αντίθετα, όταν το pixel είναι μεγαλύτερο από 255, τότε το $diff$ είναι θετικό και η μάσκα που προκύπτει είναι 0, οπότε απομένει μόνο ο όρος 255 στο αποτέλεσμα.

$$diff = x - 255, \quad x \geq 255$$

$$output_{i,j} = 255 + diff \& 0 = 255$$

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
11_sobel_fast_psnr	16.30×	0.10762	0.00150	0.104974 / 0.111273
12_sobel_fast_min	16.41×	0.10687	0.00084	0.105859 / 0.109175

Πίνακας 24: Χρόνος εκτέλεσης σε-00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
11_sobel_fast_psnr	13.18×	0.03562	0.00044	0.034291 / 0.037935
12_sobel_fast_min	11.69×	0.04017	0.00153	0.038610 / 0.044180

Πίνακας 25: Χρόνος εκτέλεσης σε-fast

3.5 Γρήγορη Square Root

Η πράξη της τετραγωνικής ρίζας είναι μια περίπλοκη και αργή αριθμητική πράξη. Βασιζόμενοι στον αλγόριθμο για την προσέγγιση της αντίστροφης τετραγωνικής ρίζας δημιουργήσαμε μια αντίστοιχη έκδοση για τον υπολογισμό της τετραγωνικής ρίζας. Θεωρούμε έναν αριθμό κινητής υποδιαστολής x με μαντίσσα M και εκθέτη E . Θα βρούμε τώρα τον δυαδικό λογάριθμο του x .

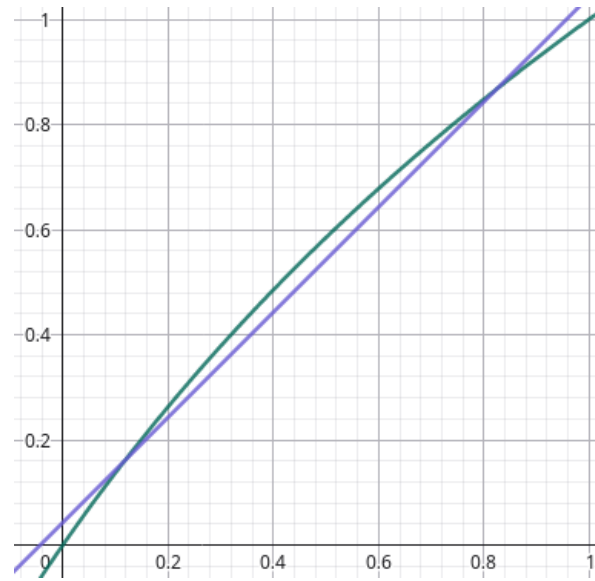
Τώρα θα πρέπει να χρησιμοποιήσουμε μια προσέγγιση της συνάρτησης $\log_2(1+x) \approx x + \mu$, από όπου προέρχεται και το σφάλμα στον υπολογισμό της τετραγωνικής ρίζας. Η επιλογή του μ γίνεται αυθαίρετα και στην περίπτωση μας επιλέξαμε $\mu = 0.043$ ώστε να έχουμε το μικρότερο σφάλμα. Συνεχίζοντας έχουμε

$$(1) \implies \frac{M}{2^{23}} + E - 127 + \mu = \frac{1}{2^{23}} (M + 2^{23} \cdot E) + \mu - 127$$

$$\log_2 x = \log_2 \left[\left(1 + \frac{M}{2^{23}} \right) \cdot 2^{E-127} \right] = \log_2 \left(1 + \frac{M}{2^{23}} \right) + E - 127 \quad (1)$$

Άρα έχουμε καταλήξει με μια σχέση η οποία εξισώνει τον αριθμό x με την αναπαράσταση του σαν αριθμός κινητής υποδιαστολής $x_f = M + 2^{23} \cdot E$.

$$\log_2 x = \frac{1}{2^{23}} x_f + \mu - 127 \quad (2)$$



Σχήμα 2: Plot of $y = x + \mu$, $\mu = 0.043$ and $y = \log_2(1 + x)$

Θεωρούμε αριθμό y για τον οποίο ισχύει $y = \sqrt{x}$. Εφαρμόζοντας λογάριθμο έχουμε

$$y = \sqrt{x} \implies \log_2 y = \frac{1}{2} \log_2 x \xrightarrow{(2)}$$

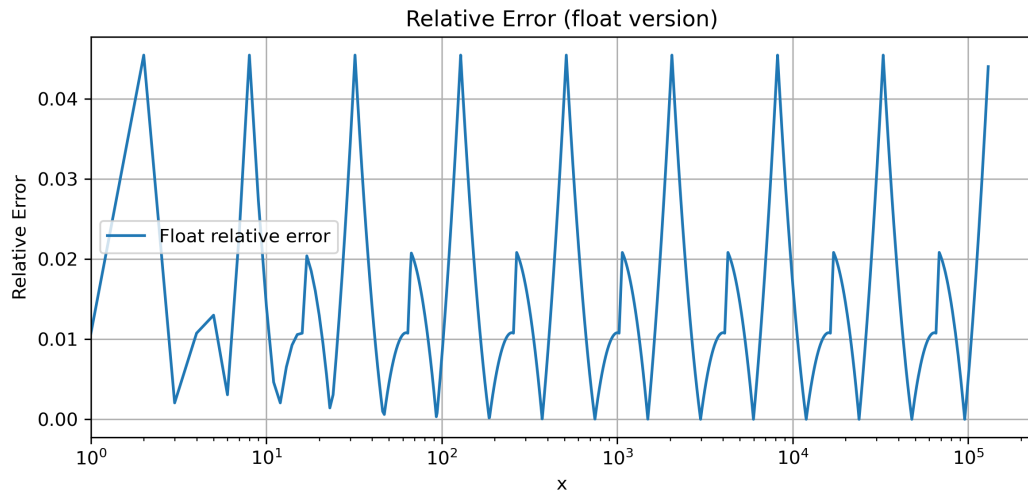
$$\frac{1}{2^{23}} y_f + \mu - 127 = \frac{1}{2^{24}} x_f + \frac{1}{2} \cdot (\mu - 127) \implies y_f = \frac{1}{2} x_f - 2^{22} \cdot (\mu - 127)$$

Επομένως διαιρώντας τον αριθμό x_f στην μορφή του ως αριθμός κινητής υποδιαστολής με το 2 και προσθέτοντας μια σταθερά παίρνουμε μια καλή εκτίμηση της τετραγωνικής ρίζας του x . Η υλοποίηση του γίνεται κάνοντας `type cast` τον αριθμό κινητής υποδιαστολής σε ακέραιο χωρίς όμως να πραγματοποιηθεί μετατροπή σε ακέραιο, δηλαδή κρατάμε απλώς τα bits του αριθμού. Ο αριθμός `0x1FBD3F7D` προκύπτει από την σταθερά $-2^{22} \cdot (\mu - 127) = 532496252.928 \approx 532496253$.

```
float fast_sqrt(float a) {
    int i = * ( int * ) &a;
    i = (i >> 1) + 0x1FBD3F7D;
    return * ( float * ) &i;
}
```

Στην έκδοση του για την αντίστροφη τετραγωνική ρίζα εφαρμόζεται και μια επανάληψη της μεθόδου Newton-Raphson που για την περίπτωση μας απαιτεί διαίρεση οπότε αποφασίσαμε να μην την εφαρμόσουμε. Αυτό οδηγεί σε μεγαλύτερα σφάλματα που όμως παραμένουν σχετικά μικρά για το εύρος των αριθμών με το οποίο ασχολούμαστε, όπως φαίνεται παρακάτω.

Παρατηρούμε πως το σφάλμα παραμένει κάτω από 5% που οδηγεί σε σχετικά ακριβή αποτελέσματα με PSNR κοντά στο 50.



Σχήμα 3: Σχετικό σφάλμα για αριθμούς από 1 μέχρι 10^5

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
12_sobel_fast_min	16.41×	0.10687	0.00084	0.105859 / 0.109175
13_sobel_fast_sqrt	18.56×	0.09451	0.00074	0.093438 / 0.096139
14_sobel_fast_conv_sqrt	17.26×	0.10160	0.00156	0.099090 / 0.114298

Πίνακας 26: Χρόνος εκτέλεσης σε -00

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
12_sobel_fast_min	11.69×	0.04017	0.00153	0.038610 / 0.044180
13_sobel_fast_sqrt	13.55×	0.03465	0.00113	0.032232 / 0.036176
14_sobel_fast_conv_sqrt	21.49×	0.02184	0.00074	0.020820 / 0.022988

Πίνακας 27: Χρόνος εκτέλεσης σε -fast

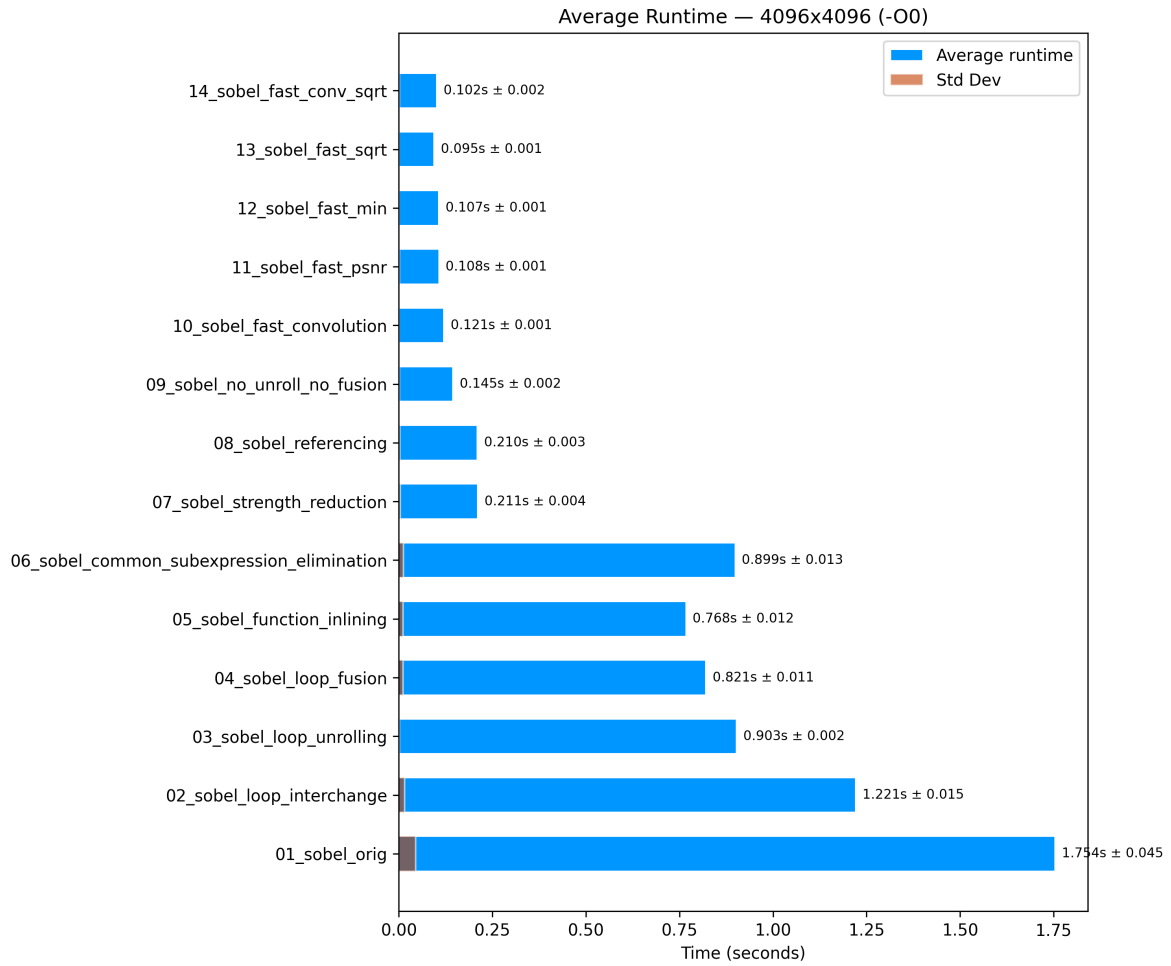
4 Αποτελέσματα

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
01_sobel_orig	1.00×	1.75385	0.04514	1.690210 / 1.916120
02_sobel_loop_interchange	1.44×	1.22102	0.01533	1.193020 / 1.259970
03_sobel_loop_unrolling	1.94×	0.90287	0.00164	0.900786 / 0.942234
04_sobel_loop_fusion	2.14×	0.82101	0.01090	0.810427 / 0.964667
05_sobel_function_inlining	2.28×	0.76785	0.01184	0.753455 / 0.798266
06_sobel_common_subexpression_elimination	1.95×	0.89913	0.01290	0.890251 / 0.958620
07_sobel_strength_reduction	8.30×	0.21133	0.00381	0.208410 / 0.226750
08_sobel_referencing	8.33×	0.21048	0.00281	0.208129 / 0.418608
09_sobel_no_unroll_no_fusion	12.08×	0.14522	0.00156	0.143731 / 0.151505
10_sobel_fast_convolution	14.53×	0.12072	0.00102	0.119427 / 0.126503
11_sobel_fast_psnr	16.30×	0.10762	0.00150	0.104974 / 0.111273
12_sobel_fast_min	16.41×	0.10687	0.00084	0.105859 / 0.109175
13_sobel_fast_sqrt	18.56×	0.09451	0.00074	0.093438 / 0.096139
14_sobel_fast_conv_sqrt	17.26×	0.10160	0.00156	0.099090 / 0.114298

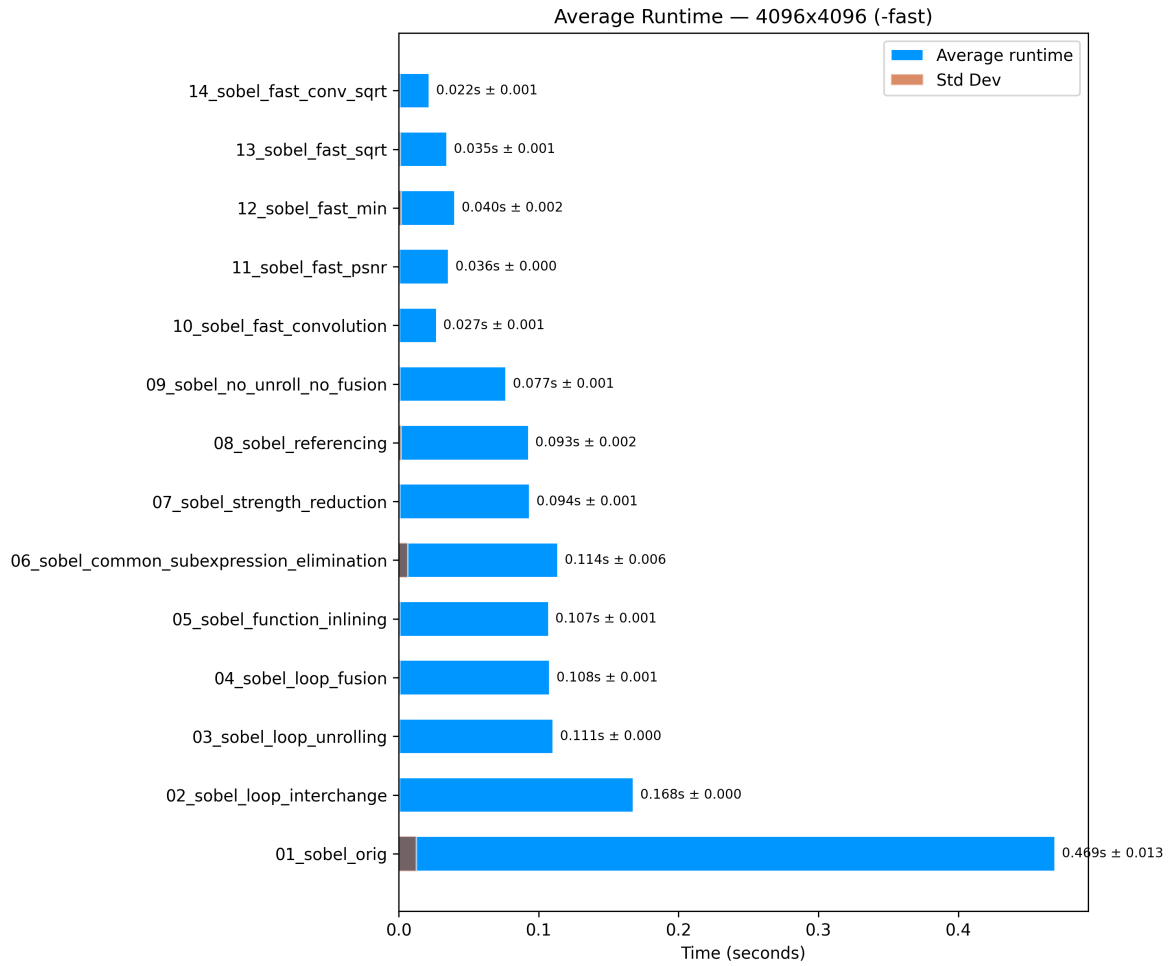
Πίνακας 28: Χρόνος εκτέλεσης σε -O0

Executable	Speedup	Average (s)	Std Dev (s)	Min / Max outliers
01_sobel_orig	1.00×	0.46939	0.01264	0.450155 / 0.518506
02_sobel_loop_interchange	2.79×	0.16796	0.00047	0.167034 / 0.177655
03_sobel_loop_unrolling	4.25×	0.11050	0.00047	0.108389 / 0.112336
04_sobel_loop_fusion	4.35×	0.10795	0.00071	0.107109 / 0.109979
05_sobel_function_inlining	4.38×	0.10729	0.00080	0.106468 / 0.119504
06_sobel_common_subexpression_elimination	4.12×	0.11386	0.00643	0.105915 / 0.162000
07_sobel_strength_reduction	5.02×	0.09359	0.00079	0.092326 / 0.095376
08_sobel_referencing	5.05×	0.09298	0.00166	0.091319 / 0.113244
09_sobel_no_unroll_no_fusion	6.11×	0.07683	0.00069	0.075577 / 0.079137
10_sobel_fast_convolution	17.30×	0.02714	0.00057	0.026333 / 0.028785
11_sobel_fast_psnr	13.18×	0.03562	0.00044	0.034291 / 0.037935
12_sobel_fast_min	11.69×	0.04017	0.00153	0.038610 / 0.044180
13_sobel_fast_sqrt	13.55×	0.03465	0.00113	0.032232 / 0.036176
14_sobel_fast_conv_sqrt	21.49×	0.02184	0.00074	0.020820 / 0.022988

Πίνακας 29: Χρόνος εκτέλεσης σε -fast



Σχήμα 4: Όλες οι εκτελέσεις σε -O0



Σχήμα 5: Όλες οι εκτελέσεις σε -fast